LU Solver on the GPU

From FP64 kernels to tensor cores, watched by NixNan

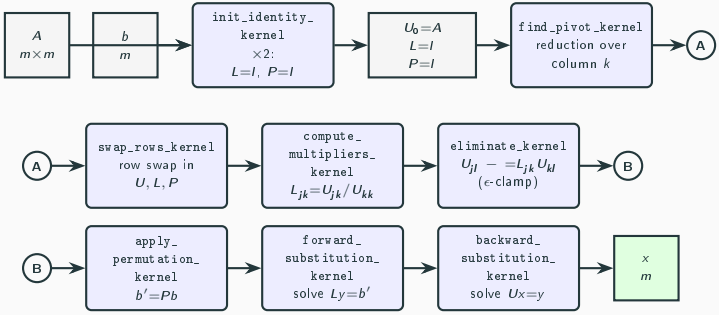
Ganesh Gopalakrishnan

SC26 tutorial

Algorithm: LU with partial pivoting, solve $Ax = b$

$$PA = LU, \qquad \underbrace{p = \arg\max_{i \geq k} |U_{ik}|}_{\text{pivot}}, \qquad \underbrace{L_{jk} = \frac{U_{jk}}{U_{kk}}}_{\text{multiplier}}, \qquad \underbrace{U_{jl} \mathrel{-}= L_{jk} U_{kl}}_{\text{eliminate, } j>k,\ l \geq k}$$

$$Ly = Pb \ \text{(forward)}, \qquad Ux = y \ \text{(backward)}$$



NixNan knobs: environment variables

`INSTR_BEGIN / END`	instr. index window	`NIXNAN_TIMEOUT`	self-terminate, secs
`TOOL_VERBOSE`	setup diagnostics	`LOGFILE`	redirect output
`ENABLE_FUN_DETAIL`	name on every rerun	`LINE_INFO`	source file:line (YMMV)
`PRINT_ILL_INSTR`	print culprit SASS	`LOG_KERNELS`	log launches only
`SAMPLING`	every *N*th launch	`MAX_ERRORS`	stop after *N* reports
`INSTRUMENT_EXCEPTIONS`	master switch	`TIME_KERNELS`	per-kernel time
`INSTR_MEM`	also check STG (excn $\rightarrow$ mem)	`FUNCTION_WHITELIST`	only these kernels
`HISTOGRAM`	exponent histogram	`FUNCTION_BLACKLIST`	all but these
`BIN_SPEC_FILE`	ranges to watch		

For example, `ksel-autonixnan` scripts (using the above env vars) the collection of kernel names and execution times, so users can then white- or black-list exactly what to observe in detail.

TIME_KERNELS: the first-launch cost

Kernel	Launches	First launch (µs)	Median later (µs)	Ratio
`init_identity_kernel`	2	312,451	21	≈14,900×
`find_pivot_kernel`	19	298,100	20	≈14,900×
`swap_rows_kernel`	19	290,205	25	≈11,600×
`compute_multipliers_kernel`	19	295,432	20	≈14,800×
`eliminate_kernel`	19	278,808	20	≈13,900×
`apply_permutation_kernel`	1	300,320	—	—
`forward_substitution_kernel`	1	326,736	—	—
`backward_substitution_kernel`	1	324,597	—	—

- First launch of every kernel: ≈280–330 ms. Later launches: ≈20–25 µs, four to five orders of magnitude less.
- The timer starts *before* NVBit rewrites the kernel's SASS (once per function) and stops after, so the first number is instrumentation cost, not kernel time.
- Kernels launched once (last three rows) only ever show this one-time cost.
- Library-heavy binaries pay far more: the cuSOLVER-based `pytorch_slogdet` run was killed at 120 s during instrumentation.